

≡ TOP 30 ≡

OOPS

— • —
BASED
INTERVIEW
QUESTIONS



TOP 30 OOPS

BASED INTERVIEW QUESTIONS

• Page 1 : OOP Fundamentals •

1 What is OOP? Why is it used?

Answer : Object Oriented Programming (OOP) is a programming paradigm based on the concept of "objects", which contain data (attributes) and code (methods). It helps in writing modular, reusable and maintainable code.

Why is it used?

- Improves code reusability
- Easy to maintain and modify
- Provides real-world modeling
- Enhances security through data hiding
- Supports large and complex applications

Real-world Example

Attributes

(color, speed, model)



Methods

(start(), stop(), accelerate())

2 What are the four pillars of OOP?

Answer : The four main pillars of OOP are:

1. Encapsulation



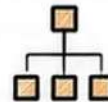
Binding data (variables) and methods that work on data into a single unit (class) and restricting direct access to some of an object's components.

2. Abstraction



Hiding the internal details and showing only essential features to the user.

3. Inheritance



Acquiring the properties and behaviors of a parent class into a child class.

4. Polymorphism



The ability of an object to take many forms. The same action can behave differently in different situations.

3 Class vs Object

Class

- A class is a blueprint or template.
- It is a logical entity.
- It cannot be allocated memory.
- Example: Student

```
class Student {
    int id;
    String name;
    void display() {
        System.out.println("Student Info");
    }
}
```

Object

- An object is an instance of a class.
- It is a real entity.
- Memory is allocated when an object is created.
- Example: s1 is an object of Student class

```
Student s1 = new Student();
s1.id = 101;
s1.name = "Aman";
s1.display(); // Output: Student Info
```

Memory

s1	
id	101
name	Aman
display() (address)	

• Page 2 : Constructors & Encapsulation •

4

What is a Constructor?

Answer : A constructor is a special method that is automatically called when an object is created. It is used to initialize the object.

Key Points

- Same name as class
- No return type (not even void)
- Called automatically
- Can be parameterized or non-parameterized

Example

```
class Student {
    int id;
    String name;
    // Constructor
    Student(int id, String name) {
        this.id = id;
        this.name = name;
    }
}
```

Output

```
Student s1 = new Student(1, "Aman");
s1.id → 1
s1.name → Aman
```

5

Constructor vs Method

Basis	Constructor	Method
1. Name	Same as class name	Can be any valid name
2. Return Type	No return type	Has return type (or void)
3. Call	Called automatically when object is created	Called explicitly
4. Purpose	To initialize the object	To perform operations
5. Inheritance	Not inherited	Inherited
6. Overloading	Yes (constructor overloading)	Yes (method overloading)

Example

```
class Demo {
    Demo() {
        System.out.println("Constructor called");
    }
}
```

```
void show() {
    System.out.println("Method called");
}
```

6

What is Encapsulation?

Answer : Encapsulation is the process of binding data (variables) and methods that operate on data into a single unit (class) and restricting direct access to some of an object's components. It helps in data hiding and security.

Key Points

- Hides internal state
- Controls access using access modifiers
- Improves security and data integrity
- Achieved using private, getter and setter

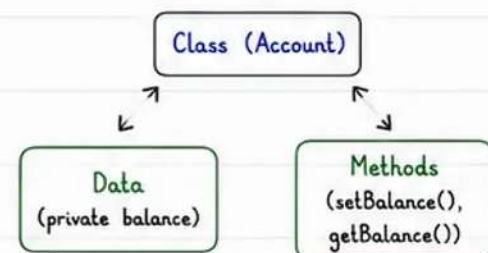
Example

```
class Account {
    private double balance;

    public void setBalance(double bal) {
        this.balance = bal;
    }

    public double getBalance() {
        return balance;
    }
}
```

How it works?



Direct access to balance is not allowed. It can be accessed only through methods.

• Page 3 : Abstraction Concepts •

7

How do you achieve Encapsulation?

Answer : Encapsulation is achieved by declaring the variables of a class as **private** and providing public getter and setter methods to access and update the values.

Key Points

- Use private access modifier for data.
- Provide public getter and setter methods.
- Prevents direct access to data.
- Improves security and control.
- Helps in data validation.

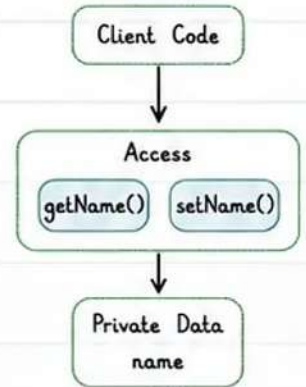
Example

```
class Student {
    private String name; // data is hidden

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }
}
```

How it works?



Data is accessed indirectly through methods.

8

What is Abstraction?

Answer : Abstraction is the process of hiding implementation details and showing only essential features (functionality) to the user.

It focuses on what an object does, not on how it does.

Key Points

- Hides complex implementation.
- Shows only necessary features.
- Achieved using abstract classes and interfaces.
- Improves reusability and maintainability.
- Helps to reduce complexity.

Example

```
// Using abstract class
abstract class Shape {
    abstract void draw(); // abstract method
}

class Circle extends Shape {
    void draw() {
        System.out.println("Drawing Circle");
    }
}
```

Real-life Analogy



You know
which button
to press
(what to do)



But you don't know
how the TV works
inside
(how it is done)

User focuses on functionality,
not on internal working.

9

Abstraction vs Encapsulation

Basis	Abstraction	Encapsulation
1. Focus	Hides implementation details and shows functionality.	Hides data and binds data with methods.
2. Purpose	To reduce complexity.	To protect data and control access.
3. Achieved By	Abstract classes, Interfaces	Access modifiers (private, protected, public etc.)
4. What it hides?	Internal working / implementation.	Data / Variables.
5. Level	Design Level	Implementation Level
6. Example	Animal class with abstract method sound().	Class with private variables and public getter/setter.

Quick Summary

- Abstraction = What an object does.
- Encapsulation = How an object protects its data.
- Both are complementary and important pillars of OOP.

Think in terms of objects, design for reuse, and code for clarity! **100**

• Page 4 : Abstract Class & Interface •

10

Abstract Class vs Interface

Answer : Both are used to achieve abstraction in Java, but they have differences in terms of implementation, variables, inheritance and usage.

Example

Basis	Abstract Class	Interface
1. Keyword	<code>abstract class</code>	<code>interface</code>
2. Methods	Can have both abstract and concrete methods.	Before Java 8 - only abstract methods. (Java 8+ can have default & static methods)
3. Variables	Can have instance variables and static variables.	Only public static final variables.
4. Constructor	Can have constructors.	Cannot have constructors.
5. Inheritance	A class can extend only one abstract class.	A class can implement multiple interfaces.
6. Access Modifiers	Can have any access modifier.	Methods are public by default.
7. Use	When we want to share common code.	When we want to define a contract / capability.

Abstract Class

```
abstract class Animal {
    String color;
    Animal(String color) {
        this.color = color;
    }
    abstract void sound();

    void sleep() {
        System.out.println("Sleeping...");
    }
}
```

Interface

```
interface Animal {
    String type = "Living Being";
    void sound();

    default void info() {
        System.out.println("Animal");
    }

    static void show() {
        System.out.println("Static method in interface");
    }
}
```



If you want to provide some common implementation → use **Abstract Class**.
If you want to define a contract → use **Interface**.

11

What is an Interface?

Answer : An interface is a blueprint of class contains a group of related methods with empty bodies (abstract methods) and constants. It is used to achieve 100% abstraction.

Key Points

- All methods are **public** and **abstract** by default.
- All variables are **public**, **static** and **final**.
- It cannot be instantiated.
- A class can implement multiple interfaces.
- Used to achieve abstraction and multiple inheritance.

Example

```
interface Drawable {
    void draw(); // abstract method
}

class Circle implements Drawable {
    public void draw() {
        System.out.println("Drawing Circle");
    }
}
```

Real-world Analogy



Switch
(Interface)



Bulb
(Class that implements the interface)



Switch defines what to do (ON/OFF), not how.
Bulb decides how to do it.

12

Why do we use Interfaces?

Answer : Interfaces are used to achieve abstraction, loose coupling and multiple inheritance in Java.

Key Reasons

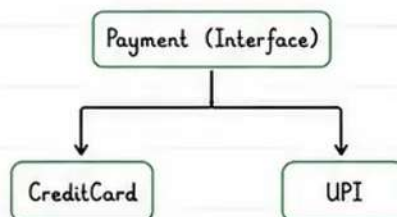
- Achieve 100% Abstraction
- Support Multiple Inheritance
- Loose Coupling
- To define a contract between the class and the outside world.
- Easy to maintain and extend.

```
interface Payment {
    void pay();
}

class CreditCard implements Payment {
    public void pay() {
        System.out.println("Paid using Credit Card");
    }
}

class Upi implements Payment {
    public void pay() {
        System.out.println("Paid using UPI");
    }
}
```

How it helps?



We can add more payment methods in future without changing the existing code.



Interfaces define a contract, classes implement the contract.
This makes code flexible, extensible and easy to maintain.

100

• Page 5 : Inheritance Basics •

13 What is Inheritance?

Answer : Inheritance is a mechanism in OOP where a new class (child or subclass) acquires the properties and behaviors (fields and methods) of an existing class (parent or superclass).

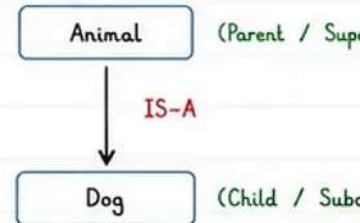
Key Points

- Promotes code reusability.
- Establishes IS-A relationship.
- Helps in method overriding (runtime polymorphism).
- Improves extensibility.
- Reduces redundancy.

Example

```
class Animal {
    void eat() { System.out.println("Eating..."); }
}
class Dog extends Animal {
    void bark() { System.out.println("Barking..."); }
}
public class Test {
    public static void main(String[] args) {
        Dog d = new Dog();
        d.eat(); // Inherited method
        d.bark(); // Own method
    }
}
```

Visual Representation (IS-A)



Dog IS-A Animal.
So, Dog inherits properties and methods of Animal.

14 What are the Types of Inheritance?

Answer : There are different types of inheritance supported by OOP languages.

Types

Type of Inheritance	Description	Diagram	Supported in Java?
1. Single	One subclass inherits from one superclass.		Yes
2. Multilevel	A subclass inherits from a superclass which is also a subclass.		Yes
3. Hierarchical	Multiple subclasses inherit from a single superclass.		Yes
4. Multiple	A subclass inherits from multiple superclasses.		No (through classes) Yes (using Interfaces)
5. Hybrid	Combination of two or more types.		No (through classes) Yes (using Interfaces)

Note



Java supports Single, Multilevel and Hierarchical inheritance using classes. Multiple & Hybrid are achieved using Interfaces. (We will see more in next pages)

15 Why doesn't Java support Multiple Inheritance through Classes?

Answer : Java does not support multiple inheritance through classes to avoid ambiguity (also known as the "Diamond Problem").

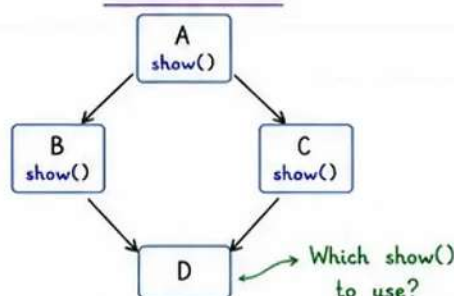
If a class inherits from two classes having the same method, the JVM gets confused about which method to use.

Example of Ambiguity

```
class A { void show() { System.out.println("A"); } }
class B { void show() { System.out.println("B"); } }

// If class C extends A and B,
// show() from A or B ? -> Ambiguity!
```

Diamond Problem



How Java solves this?

- Java allows multiple inheritance using Interfaces.
- In interfaces, methods are abstract by default (no implementation).
- So, no ambiguity.



Rule: Use Interfaces for multiple inheritance, not classes.
That's why Java is simple and so

• Page 6 : Inheritance & Method Overriding •

16

extends vs implements

Answer : Both are used to achieve inheritance in Java, but they are used in different scenarios.

Basis	extends (Class)	implements (Interface)
1. Purpose	Used to inherit a class.	Used to implement an interface.
2. Relationship	IS-A (Inheritance)	IS-A (Abstraction)
3. Multiple	A class can extend only one class.	A class can implement multiple interfaces.
4. Keyword	extends	implements
5. Methods	Inherits concrete and abstract methods.	Implements only abstract methods (before Java 8).
6. Constructor	Constructors are inherited.	No constructors in interfaces.
7. Usage	Code reusability.	Achieve 100% abstraction and multiple inheritance.

Example

```
// Using extends
class Animal {
    void eat() { System.out.println("Eating"); }
}

class Dog extends Animal {
    void bark() { System.out.println("Barking"); }
}

class Test {
    public static void main(String[] args) {
        Dog d = new Dog();
        d.eat(); // inherited
        d.bark(); // own method
    }
}
```

```
// Using implements
interface Shape {
    void draw();
}

class Circle implements Shape {
    public void draw() {
        System.out.println("Drawing Circle");
    }
}

class Test {
    public static void main(String[] args) {
        Shape s = new Circle();
        s.draw();
    }
}
```



Use **extends** for IS-A (class to class).

Use **implements** for IS-A (class to interface).

17

What is Method Overriding?

Answer : Method overriding is a feature in Java where a subclass provides a specific implementation of a method that is already defined in its parent class. It is used to achieve runtime polymorphism.

Key Points

- Occurs in inheritance.
- Must have same method name and parameters.
- Must have same or covariant return type.
- Cannot reduce the visibility of the method.
- Achieved at runtime (Dynamic Binding).

Example

```
class Animal {
    void sound() {
        System.out.println("Animal makes sound");
    }
}

class Dog extends Animal {
    @Override
    void sound() {
        System.out.println("Dog barks");
    }
}

class Test {
    public static void main(String[] args) {
        Animal a = new Dog();
        a.sound(); // Dog barks
    }
}
```

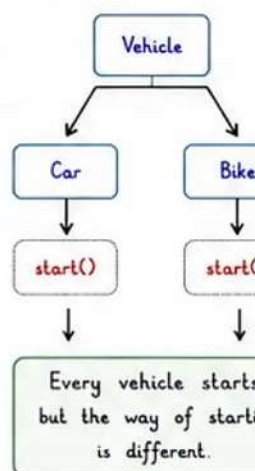
How it works?

```
Animal a = new Dog();
```

```
a.sound();
```

At runtime, JVM finds the actual object (Dog) and calls Dog's sound() method.

Real-life Analogy



18

What are the Rules of Method Overriding?

Answer : To override a method in Java, certain rules must be followed.

Rules

- The method must have the same name.
- The method must have the same parameters.
- The method must have the same return type or covariant return type.
- The access modifier in subclass must be same or broader, not narrower.
- The method cannot be private, static or final.
- The method must not throw broader checked exceptions.

Example (Valid Overriding)

```
class Parent {
    protected Number getValue() {
        return 10;
    }
}

class Child extends Parent {
    @Override
    public Integer getValue() { // Covariant return type
        return 20;
    }
}
```

Example (Invalid Overriding)

```
// 1. Different parameter
class A { void show(int a) {} }
class B extends A { void show(int a, int b) {} }

// 2. Narrower access modifier
class A { public void show() {} }
class B extends A { private void show() {} }

// 3. Static method
class A { static void show() {} }
class B extends A { void show() {} } // ✗
```



Overriding ensures runtime polymorphism. The actual object decides which method to execute.

100

Page 7 : Polymorphism Basics

19

What is Polymorphism?

Answer: Polymorphism means "many forms". It allows objects of different classes to be treated as objects of a common superclass. The actual behavior is decided at runtime.

Example

```
class Animal {
    void sound() {
        System.out.println("Animal sound");
    }
}

class Dog extends Animal {
    @Override
    void sound() {
        System.out.println("Bark");
    }
}

Animal a = new Dog();
a.sound(); // Output: Bark (runtime polymorphism)
```

Animal a = new Dog();

a.sound()

Dog's sound() is executed
(at runtime)



Reference type = Animal
Object type = Dog
Method called = Dog's version

Key Points

- One interface, multiple implementations.
- Achieved using method overriding (runtime) and method overloading (compile-time).
- Increases flexibility and extensibility.

20

Compile-time vs Runtime Polymorphism

	Compile-time Polymorphism	Runtime Polymorphism
Also called	Static Polymorphism	Dynamic Polymorphism
Achieved by	Method Overloading	Method Overriding
Binding	Early Binding	Late Binding
When resolved	At compile time	At runtime
Example	sum(int a, int b) and sum(int a, int b, int c)	sound() in Animal and Dog
Performance	Faster	Slightly slower (due to runtime lookup)

Quick View

Compile-time
(Overloading)

Same method name
Different parameters

Resolved at
compile time

Runtime
(Overriding)

Same method name
Same parameters

Resolved at
runtime

21

What is Method Overloading?

Answer: Method overloading means having multiple methods in the same class with the same name but different parameters (number, type, or order).

Example

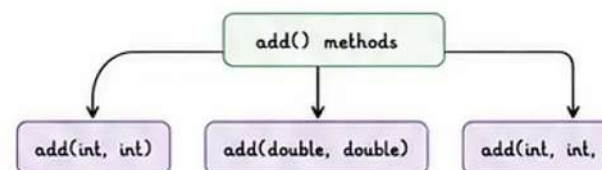
```
class Calculator {
    int add(int a, int b) {
        return a + b;
    }
    double add(double a, double b) {
        return a + b;
    }
    int add(int a, int b, int c) {
        return a + b + c;
    }
}

Calculator c = new Calculator();
System.out.println(c.add(2, 3)); // 5
System.out.println(c.add(2.5, 3.5)); // 6.0
System.out.println(c.add(1, 2, 3)); // 6
```

Rules

- Same method name.
- Different parameter list.
- Can change number, type, or order of parameters.
- Return type can be same or different.
- It is compile-time polymorphism.

Visual



The compiler decides which method to call based on the arguments passed.



Remember: Overloading is about having many methods.
Overriding is about changing the behavior of an existing method.

Page 8 : Overloading vs Overriding & Runtime Behavior

22 Method Overloading vs Method Overriding

Method Overloading	Method Overriding
- Same method name, different parameters (type/number/order). - Happens in the same class. - Achieved at compile-time (Static Polymorphism). - Return type can be same or different (but not only return type).	- Same method name, same parameters. - Happens in child class. - Achieved at runtime (Dynamic Polymorphism). - Return type must be same (or covariant).

Quick Note :

Overloading → same name, different si

Overriding → same sign, different clo



Overloading is about having many methods with the same name.

Overriding is about changing the behavior of parent method.

23 Can a method be overloaded by changing only its return type?

Answer : No. Changing only return type is not sufficient for overloading.

Why?

Because return type is not part of method signature.

Two methods must have different parameters.

Key Point :

Method signature =

Method Name + Parameter list

(Return type is **NOT** included)

✗ Invalid Overloading

```
class Test {
    int show() { return 10; }
    // Error: same parameters, only return type different
    void show() { System.out.println("Hello"); }
}
```

✓ Valid Overloading

```
class Test {
    int show() { return 10; }
    void show(int a) { System.out.println(a); }
    int show(int a, int b) { return a + b; }
}
```

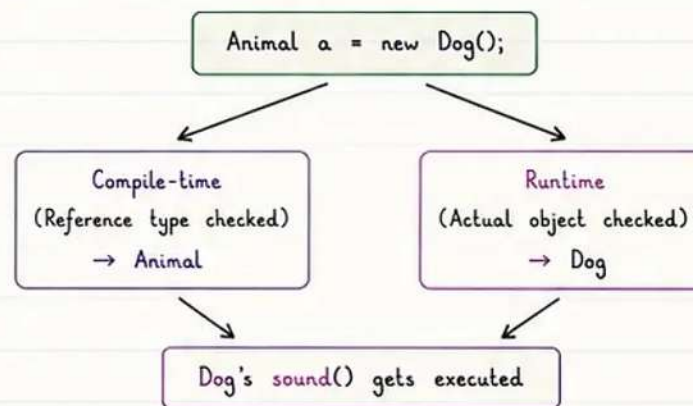
24 What is Dynamic Method Dispatch?

Answer : Dynamic Method Dispatch is the process by which a call to an overridden method is resolved at runtime, not at compile time.

Example :

```
class Animal {
    void sound() { System.out.println("Animal sound"); }
}
class Dog extends Animal {
    @Override
    void sound() { System.out.println("Dog barks"); }
}
class Test {
    public static void main(String[] args) {
        Animal a = new Dog(); // reference type = Animal
        a.sound(); // Output: Dog barks (resolved at runtime)
    }
}
```

How it works?



JVM looks at the actual object (Dog) at runtime to decide which method to call.

Page 9 : Abstraction & Encapsulation

25 What is Abstraction?

Answer : Abstraction is the process of hiding implementation details and showing only essential features to the user.

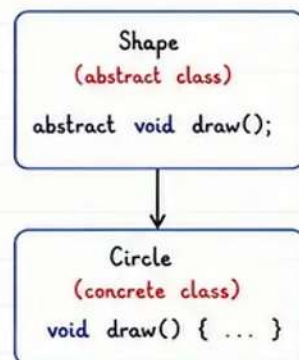
Key Points

- Focus is on WHAT an object does, not HOW it does.
- Achieved using abstract classes and interfaces.
- Helps in reducing complexity.

Example (Abstract Class)

```
abstract class Shape {
    abstract void draw(); // abstract method
}
class Circle extends Shape {
    void draw() {
        System.out.println("Drawing Circle");
    }
}
class Test {
    public static void main(String[] args) {
        Shape s = new Circle();
        s.draw();
    }
}
```

Visual



★ Abstraction = Show important things to the user, hide the unnecessary details.

26 What is Encapsulation?

Answer : Encapsulation is the wrapping of data (variables) and code (methods) together into a single unit (class) and restricting direct access to some of the object's components.

Key Points

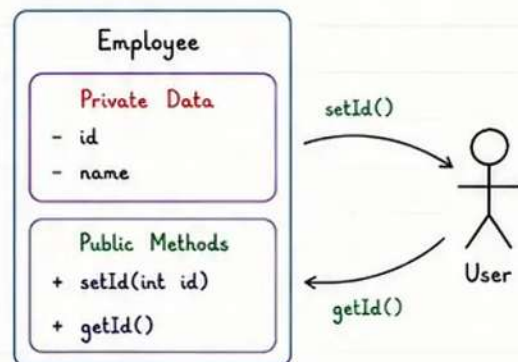
- Protects data from outside interference.
- Achieved using private variables and public getter/setter methods.
- Improves security and data hiding.

Example

```
class Employee {
    private int id;
    private String name;

    public void setId(int id) {
        this.id = id;
    }
    public int getId() {
        return id;
    }
}
```

Visual



★ Encapsulation = Bind data + methods together and protect the data.

27 Difference between Abstraction and Encapsulation?

Answer :

Feature	Abstraction	Encapsulation
Purpose	Hiding implementation details	Hiding data and controlling access
Focus	On what an object does	On how data is accessed
Achieved using	Abstract classes, Interfaces	Private variables, Getter/Setter
Level	Design level	Implementation level
Example	abstract void draw();	private int id; public int getId();

Quick Tip

💡 Abstraction is about showing only essential things.

Encapsulation is about protecting the data and controlling access.



Together, Abstraction + Encapsulation = Secure, Flexible and Maintainable Code.

Page 10 : Summary & Key Takeaways

28

Rules for a Good Design (Java)

1. Follow SOLID principles.
2. Keep the design simple and understandable.
3. Program to interfaces, not implementations.
4. Use abstraction to reduce complexity.
5. Use encapsulation to protect data.
6. Prefer composition over inheritance.
7. Write clean, maintainable and reusable code.

SOLID Principles (Quick View)

- S = Single Responsibility Principle
 O = Open/Closed Principle
 L = Liskov Substitution Principle
 I = Interface Segregation Principle
 D = Dependency Inversion Principle



Good design makes the code easy to change, test and extend.

29

OOPs Concepts - Quick Revision

Concept	What is it?	Example
Class	Blueprint for objects	<code>class Car { ... }</code>
Object	Instance of a class	<code>Car c = new Car();</code>
Abstraction	Show essentials, hide details	<code>abstract class Shape</code>
Encapsulation	Bind data and methods, restrict access	<code>private data + public methods</code>
Inheritance	One class acquires properties of another	<code>class Car extends Vehicle</code>
Polymorphism	One interface, many implementations	<code>method overloading/overriding</code>

Remember



All these concepts work together to make Java powerful, flexible and scalable.

30

Top 30 OOPs Questions Covered

- | | | |
|------------------------------------|---|---|
| 1. What is OOPs? | 11. Method Overloading? | 21. What is Method Overloading? |
| 2. What are the 4 pillars of OOPs? | 12. Method Overriding? | 22. Overloading vs Overriding? |
| 3. What is Class? | 13. Overloading vs Overriding? | 23. Change return type for Overloading? |
| 4. What is Object? | 14. What is Runtime Polymorphism? | 24. What is Dynamic Method Dispatch? |
| 5. What is Abstraction? | 15. What is Dynamic Method Dispatch? | 25. What is Abstraction? |
| 6. What is Encapsulation? | 16. Extends vs Implements? | 26. What is Encapsulation? |
| 7. What is Inheritance? | 17. What is Method Overriding? | 27. Abstraction vs Encapsulation? |
| 8. Types of Inheritance? | 18. Rules of Method Overriding? | 28. Rules for Good Design? |
| 9. What is Polymorphism? | 19. Polymorphism Basics? | 29. OOPs Concepts - Quick Revision? |
| 10. Types of Polymorphism? | 20. Compile-time vs Runtime Polymorphism? | 30. Summary & Key Takeaways? |

Final Takeaway



Understand the concepts deeply, write code, and practice more.
 That's the key to master OOPs in Java! 😊

Keep Learning. Keep Practicing. All the Best! ❤️

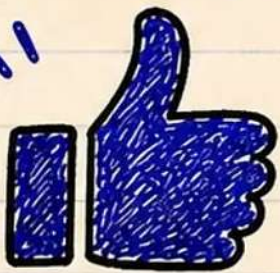
COMMENT

PDF

TO GET PDF NOTES

@crafted_note

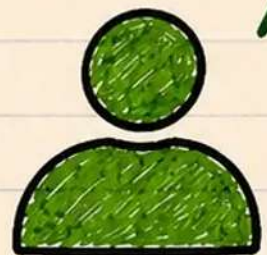
♥ FOLLOW for more useful notes ♥



LIKE



SHARE



FOLLOW



Save this post for later

